Deston Willis

Dr. Forouraghi

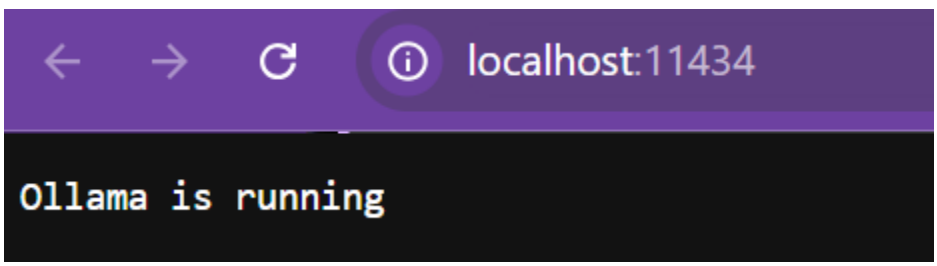
Independent Research Project

20 March 2026

Report 1

I started by following the instructions in the Module 4 overview to set up the Ollama server. After downloading the Ollama app and installing all of the necessary dependencies I started the server and ran the python script Local_Model_Basic.py located in the class Github in Module 4.

Message After Server is Started:



Output:

```
C:\Users\Deston\PycharmProjects\PythonProject\.venv\Scripts\python.exe C:\Users\Deston\PycharmProjects\PythonProject\Local_Model_Basic.py
LangChain stands out as a powerful tool for developing chatbots using language models (LLMs) due to several key features and flexibility:

1. **Customizable Models**: LangChain enables users to replace the base model with any available LLM from Hugging Face or PyTorch, offering unpa

2. **Command-Line Interface**: The standard command-line interface provides a consistent experience across different chatbots, ensuring uniform

3. **Model Extension**: Users can modify parts of the model with custom code, allowing for nuanced and specialized interactions that go beyond c

4. **PyTorch Integration**: Leveraging PyTorch's extensive ecosystem, LangChain offers advanced features like fine-tuning models, enabling more

5. **Consistent Experience**: The standard interface ensures a familiar interaction experience, enhancing usability and consistency across diffe

6. **Scalability**: Easily deployable across various platforms and frameworks, making it accessible for both enterprise and individual users.

7. **Community Support**: As an open-source project with active contributors, LangChain benefits from continuous improvement and new features ba

8. **Learning Curve**: The flexibility in model customization and command-line interface make it easier to set up compared to some other chatbot

In summary, LangChain's flexibility, integration capabilities, and open-source nature make it an excellent choice for developing adaptable chatb

Process finished with exit code 0
```

Next I installed the necessary dependencies to run a Streamlit app. I then used the streamlit hello command to start a Streamlit app. After this I created the File_App python script. I used the app.py found in the class Github in Module 4 as the starting point for the script and then edited it. In order to have the app display the files of a chosen directory I placed a text box in the sidebar for the user to enter in a directory path. Once a path is entered, the code checks if the path is valid. If it is valid, then the files in the path are loaded into the vector space. If it is not valid, then an error message saying that the path isn't valid will be displayed. Note that this message will be displayed until a valid path is given. If there are any errors reading the files, then an error is thrown saying that there was an error reading some of the files. If the directory's files can be read successfully, then the contents in the directory will be displayed in the sidebar.

Code for reading directory and its contents:

```python

# Sidebar for managing documents
with st.sidebar:
    st.header("Upload Directory")
    uploaded_directory_path = st.text_input("Insert a Directory Path")
    # Check if directory is a valid path
    if os.path.isdir(uploaded_directory_path):
        uploaded_directory = DirectoryLoader(path=uploaded_directory_path, recursive=True)
        if uploaded_directory:
            # Will load the files and display the directory contents if
            # All the files can be read successfully
            try:
                documents = uploaded_directory.load()
                vector_store.add_documents(documents)
                st.success("Valid Directory Uploaded")
                st.text("Directory Contents:")
                st.text(os.listdir(uploaded_directory_path))
            # Exception is thrown if there is a file that can't be read
            except Exception as e:
                st.error("There was an error reading some files in this directory")
    else:
        st.error("No Valid Directory Path")

```

Output of Successfully Reading Directory and Listing Contents:

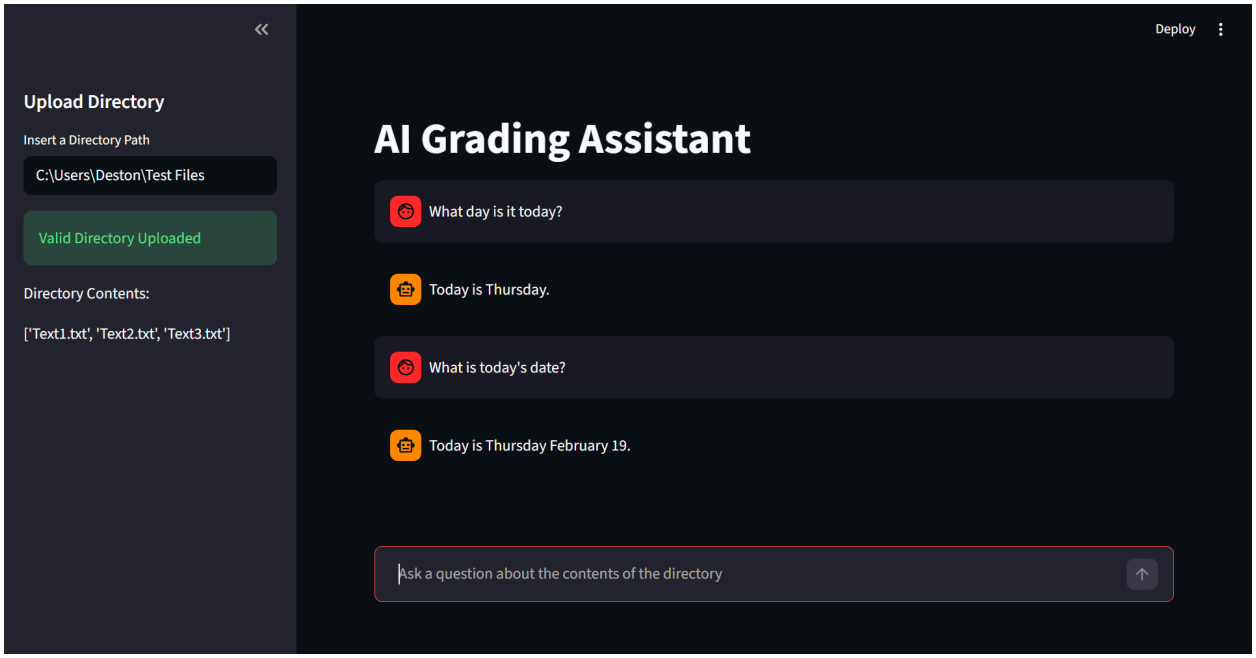
Output of Not Finding Valid Directory Path

Output of Error Reading Some Files

So far I haven't changed much with the LangGraph logic from the base code. I got rid of the auditor but kept everything else the same. This is the LangGraph code:

```python
# --- 2. STATE & GRAPH LOGIC ---
class State(TypedDict):
    messages: List
    context: str
    answer: str

def retrieve(state: State):
    """Librarian: Pulls the right facts from your documents."""
    query = state["messages"][-1].content
    docs = vector_store.similarity_search(query, k=2)
    context_text = "\n\n".join([d.page_content for d in docs])
    return {"context": context_text}

def generate(state: State):
    """Writer: Gemma-2b drafts a professional response."""
    prompt = f"Using ONLY this context: {state['context']}, answer: {state['messages'][-1].content}"
    response = writer_llm.invoke(prompt)
    return {"answer": response.content}

# Building the workflow: Search -> Write -> Done
builder = StateGraph(State)
builder.add_node("retrieve", retrieve)
builder.add_node("generate", generate)

builder.add_edge(START, "retrieve")
builder.add_edge("retrieve", "generate")
builder.add_edge("generate", END)
graph = builder.compile()
```

I then tested the model. To do so, in one of the test files that were read, it says that the date is Thursday, February 19. I then asked the model what day it is today and today's date is. This is the output:


Deploy
Upload Directory
Insert a Directory Path
C:\Users\Deston\Test Files
Valid Directory Uploaded
Directory Contents:
['Text1.txt', 'Text2.txt', 'Text3.txt']
AI Grading Assistant
What day is it today?
Today is Thursday.
What is today's date?
Today is Thursday February 19.
Ask a question about the contents of the directory